

Plan: Hybrid α -Blended Top- K Cycle Scorer (Signal $\times$ Diversity Pareto Frontier)

2026-05-10

Motivation

Two prior plans exposed two endpoint heuristics for HSiKAN's M_e cycle source on Epinions:

- **Global ABB** (`enumerate_top_k_cycles_par_bb` + `FractionNegativeScorer`): selects the highest-fraction-negative cycles. Wall: ~ 5 s. AUC: 0.5755 (single seed). All cycles are score-1.0 all-negative balanced; M_e has no score variance and biased vertex coverage.
- **Entropy heuristic** (`enumerate_top_k_cycles_par_entropy` + `EntropyGainScorer`): selects cycles to maximise per-vertex incidence-distribution entropy. Wall: ~ 17 s. AUC: 0.5396 (single seed). Vertex-uniform but signal-poor: F_1 collapses to 0.27 because retained cycles don't carry class-discriminating sign structure.

The per-vertex baseline at AUC 0.6416 achieves both: vertex-uniform *and* high-signal-per-vertex, at the cost of ~ 110 s enumeration wall on Epinions $k=4$.

This plan proposes the natural hybrid: a single scorer

$$\text{score}_\alpha(c \mid C) = \alpha \cdot \text{signal}(c) + (1 - \alpha) \cdot \Delta H(c \mid C),$$

parameterised at runtime by $\alpha \in [0, 1]$, with $\alpha=0$ recovering pure entropy and $\alpha=1$ recovering pure ABB. The intermediate sweep maps the *signal $\times$ diversity Pareto frontier* on real workloads the empirical question is whether *any* interior α recovers per-vertex-baseline AUC at ABB-class wall time.

Scope and goal

In scope:

- New trait-blanket impl `HybridScorer<B, H>` parameterised by:
 - **B**: `BoundedScorer` the signal component (existing `FractionNegativeScorer` / `BalanceScorer` / `SignProductAbsScorer` / `LowRootScorer`).
 - **H**: `UniformityHeuristic` the diversity component (existing `EntropyGainScorer` or `InverseDegreeScorer`).
 - $\alpha \in [0, 1]$ runtime mixing weight.

Implements `UniformityHeuristic` (the richer trait, since the diversity term is state-dependent); state-update / rollback delegate to the H component.

- Admissible UB by *linearity of upper bounds*:

$$\text{UB}_\alpha = \alpha \cdot \text{UB}_{\text{signal}} + (1 - \alpha) \cdot \text{UB}_{\text{div}}.$$

If both component UBs are admissible (the prior plans assert this), the linear combo is admissible by Jensen's inequality.

- No new entry point reuse `enumerate_top_k_cycles_par_entropy` (already takes a `UniformityHeuristic`) `HybridScorer` *is* a `UniformityHeuristic`.
- PyO3 binding extension: add `alpha: f64` parameter to `enumerate_top_k_cycles_signed_entropy_rs` (default $\alpha = 0$ preserves existing behaviour) and a `signal_kind` parameter selecting the B type.
- Python route: `HSIKAN_TOPK_HYBRID_ALPHA` env-var (defaults to 0 = pure diversity); `HSIKAN_TOPK_SIGNAL` selects the signal scorer.

Out of scope:

- Per-vertex hybrid (the per-vertex pipeline already mixes signal + uniformity by construction; this plan is for the global-K variants).
- Search over α inside the enumerator (we sweep α *outside* the call as a hyperparameter; no inner-loop optimisation).
- Degree-adaptive m_v that's a separate plan (the original plan B framing).

Affected files

- `hymeko_graph/src/topk_cycles.rs` additive: `HybridScorer<B, H> struct + UniformityHeuristic impl + 1–2 helper constructors`. ~ 100 LOC.
- `hymeko_graph/src/lib.rs` re-export `HybridScorer`.
- `hymeko_graph/tests/hybrid_topk.rs` new integration: parity at $\alpha=0$ vs entropy, $\alpha=1$ vs ABB; UB admissibility for blended scorer; mixed- α output cardinality + structural sanity.
- `hymeko_py/src/cycles.rs` extend `enumerate_top_k_cycles_signed_entropy_rs` with `alpha + signal_kind` args (additive defaults preserve current behaviour).
- `signedkan_wip/src/n_tuples.py` read `HSIKAN_TOPK_HYBRID_ALPHA + HSIKAN_TOPK_SIGNAL`, pass to the new PyO3 args.
- `signedkan_wip/experiments/run_hybrid_alpha_sweep_2026_05_10.sh` new orchestration: 5-point α -sweep $\{0.0, 0.25, 0.5, 0.75, 1.0\}$ at the prior smoke config (c3+c4, h=4, 20 epochs, K=10 000, balance pruner, signal=fraction_negative).

CORE.YAML items touched

None. All edits in `hymeko_graph`, `hymeko_py`, or `signedkan_wip`; none in lockdown list. `tests/**`, `benches/**`, `examples/**`, `signedkan_wip/experiments/**` all in allowlist. No new dependencies.

Interface changes

```
/// Linear blend of a signal scorer and a diversity heuristic.
///
/// 'score(c, state) = alpha * signal(c) + (1 - alpha) * H_div(c, state)'
///
/// alpha in [0, 1]; alpha = 1 collapses to pure-signal global ABB,
/// alpha = 0 collapses to the pure-diversity entropy / inverse-degree
/// path. Intermediate alpha maps the score x diversity Pareto frontier.
```

```

///
/// # Admissibility
///
/// UB_hybrid(prefix, k_rem, state) = alpha * UB_signal(prefix, k_rem)
///   + (1 - alpha) * UB_div(prefix, k_rem, state).
///
/// Linear combinations of admissible UBs are admissible: for any
/// realisable completion c of the partial state,
///   score_hybrid(c) = alpha * signal(c) + (1 - alpha) * H_div(c)
///                   <= alpha * UB_signal      + (1 - alpha) * UB_div
///                   = UB_hybrid.
pub struct HybridScorer<B, H>
where
    B: BoundedScorer,
    H: UniformityHeuristic,
{
    pub signal: B,
    pub diversity: H,
    pub alpha: f64,
}

impl<B, H> HybridScorer<B, H>
where
    B: BoundedScorer,
    H: UniformityHeuristic,
{
    pub fn new(signal: B, diversity: H, alpha: f64) -> Self {
        debug_assert!((0.0..=1.0).contains(&alpha));
        Self { signal, diversity, alpha }
    }
}

impl<B, H> UniformityHeuristic for HybridScorer<B, H>
where
    B: BoundedScorer,
    H: UniformityHeuristic,
{
    fn score(&self, vs: &[u32], signs: &[i8],
             state: &UniformityState) -> f64 {
        let n_neg = signs.iter().filter(|&s| s < 0).count();
        let s_signal = self.signal.score(vs, signs);
        let s_div = self.diversity.score(vs, signs, state);
        self.alpha * s_signal + (1.0 - self.alpha) * s_div
    }

    fn update(&self, vs: &[u32], signs: &[i8],
             state: &mut UniformityState) {
        // Only the diversity component owns state; signal is stateless.
        self.diversity.update(vs, signs, state);
    }

    fn rollback(&self, vs: &[u32], signs: &[i8],
             state: &mut UniformityState) {
        self.diversity.rollback(vs, signs, state);
    }

    fn upper_bound(&self, prefix_vs: &[u32], k_remaining: usize,
                  k_len: usize, state: &UniformityState) -> f64 {
        // Component bounds. The signal UB is sign-conditional (it

```

```

    // takes n_neg_so_far as an arg). At the partial-path
    // boundary the worst case for 'signal' is "all remaining
    // edges negative" (n_neg_so_far = k_len - k_remaining for
    // fraction_negative), which is what BoundedScorer::upper_bound
    // delivers. We pass n_neg = 0 here so the signal UB is
    // *fully optimistic* about future negative edges (admissible).
    let ub_signal = self.signal.upper_bound(0, k_len, k_len);
    let ub_div = self.diversity.upper_bound(
        prefix_vs, k_remaining, k_len, state,
    );
    self.alpha * ub_signal + (1.0 - self.alpha) * ub_div
}
}

```

Note on the signal UB. The current `BoundedScorer::upper_bound(n_neg_so_far, k_remaining, k_len)` takes *partial-path* sign info, but `HybridScorer's UniformityHeuristic::upper_bound` doesn't have access to the signs of the prefix's edges (the trait passes `prefix_vs` only). Approach: pass $n_{\text{neg}} = 0$ at call time *loose but admissible* so the signal UB is the loosest possible (best case = all remaining edges negative). The DFS body can be extended in a follow-up pass to thread n_{neg} through `dfs_entropy`; deferred so this plan stays minimal.

Test strategy

Unit (in `topk_cycles.rs::tests`)

- **Boundary collapse:** at $\alpha = 1$, `HybridScorer's` `score`, `update`, `rollback`, `upper_bound` all collapse to the signal scorer's contribution alone (diversity term has weight 0). At $\alpha = 0$, they collapse to the diversity scorer alone. A 6-vertex hand fixture exercises both endpoints.
- **Linearity of UB:** numerical check on 100 random states + cycles measure $|\text{UB}_\alpha - (\alpha \cdot \text{UB}_1 + (1 - \alpha) \cdot \text{UB}_0)| < 10^{-9}$.

Integration (`tests/hybrid_topk.rs`)

- $\alpha = 0$ **parity vs entropy:** on a 200-vertex Erds–Rényi fixture, `enumerate_top_k_cycles_par_entropy` called with `HybridScorer{ alpha: 0, ... }` produces the same canonical cycle set (within tie-tolerance) as called with `EntropyGainScorer` directly.
- $\alpha = 1$ **parity vs ABB:** on the same fixture, `HybridScorer{ alpha: 1, ... }` produces the same canonical cycle SET as `enumerate_top_k_cycles_par_bb(&FractionNegativeScorer)`. Tie-breaking may differ (the entropy enumerator's heap uses different update semantics), so test asserts $\geq 90\%$ canonical-cycle agreement.
- **Output cardinality:** at $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ on a 150-vertex fixture, output size $\leq K$ exactly.
- **Admissibility on random fixtures:** 100 random partial-path states $\times$ 5 α values, every reachable completion's score $\leq \text{UB}$.

Performance (informational only)

No asserted numerical budget. Criterion bench on a 5 000-vertex synthetic graph at the 5 α values; report median + IQR + worst. Goal is to characterise wall-time-vs- α trade-off, not enforce it.

Smoke gate (single-seed Epinions)

Run the abbreviated config (c3+c4, h=4, 20 epochs, K=10 000, `balance` pruner, `signal_kind=fraction_negati` at $\alpha \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$. For each α , report total wall + AUC + macro- F_1 .

Promotion criterion: if any α achieves AUC within ± 0.02 of the per-vertex baseline (0.6416 on Epinions abbreviated config) i.e. $\text{AUC} \geq 0.6216$ promote that α to a 5-seed paired validation per the `n-seed-before-paper-headline` memory rule.

Fallback: if no α in the 5-point sweep clears the band, the plan halts at the smoke gate. Two fallback options documented in the report:

- Finer α grid (say 11-point) possible but unlikely to matter if the 5-point trend is monotonic and below the band.
- Pivot to degree-adaptive m_v (the original plan B framing) the per-vertex path's structure is what HSiKAN actually wants; speed it up rather than try to mimic it from global-K side.

5-seed paired validation gate (only if smoke passes)

Production config (c2,c3,c4,c5,w2,w3, h=4, 80 epochs, `balance` pruner) on Epinions, 5 seeds. Acceptance:

- Paired-mean AUC for the chosen α within ± 0.005 of per-vertex $m=128$ baseline (0.846 ± 0.011 from the prior `project_epinions_edge_cr_null` memory).
- Paired- σ within $\pm 1.0\sigma$ of the baseline.
- Wall-time win documented (target $\geq 5\times$ over per-vertex enumeration on the production config).

Performance budget

Metric	Per-vertex baseline	Hybrid budget	Source
1-seed Epinions wall (full pipeline)	148.6 s	≤ 60 s for at least one α	smoke gate
1-seed Epinions AUC (c3+c4, h=4, 20 ep)	0.6416	≥ 0.6216 for at least one α	smoke gate
5-seed Epinions paired AUC (production)	0.846	within ± 0.005 at chosen α	promotion gate
Output cardinality	$= K$	$= K$ at every α	test
16 GB \$4 cap	comfortably under	comfortably under	

Rollback path

Strictly additive: `HybridScorer` is a new generic struct; the four existing `BoundedScorer` impls and the two `UniformityHeuristic` impls are unchanged. `HybridScorer` *is* a `UniformityHeuristic`, so it plugs into the existing `enumerate_top_k_cycles_par_entropy` entry point without a new enumerator.

Rollback: single-revert restores prior state. No call sites break. PyO3 default values preserve the existing `enumerate_top_k_cycles_signed_entropy_rs` behaviour at $\alpha = 0 + \text{signal_kind}$ -irrelevant.

Halt conditions (CLAUDE.md §10):

- Admissibility test fails on any α revert, fix the UB derivation, re-test before re-trying.
- Smoke gate fails at every α ship as research tool; pivot to degree-adaptive m_v plan.
- 5-seed paired gate fails ship as research tool; document smoke vs full-config gap.

Architecture diagram

`plan.tikz` shows the score composition; `plan.mmd` shows the call sequence.

